

NETWORK PROGRAMMING LABORATORY

27 June 2022

Exercise

We aim at implementing a simple monitoring system for TCP flows which consists of a centralized *collector* (client) and multiple *probes* (clients) to be deployed at possibly different vantage points.

Traffic probe. By using the callback mechanism provided by the `pcap` library, the probe captures only TCP packets from a network interface/file (specified from the command line). For every packet carrying a TCP segment, the probe:

- collects the header field values: SOURCE IP, DESTINATION IP, SOURCE PORT, DESTINATION PORT, SEGMENT LENGTH
- prepares a report object with fields: `src_ip`, `dst_ip`, `src_prt`, `dst_prt`, `length`
- encodes the above object by means of JSON, with the corresponding keys
- serializes the JSON object and sends it to a UDP collector server listening on a given IP address and port (arbitrarily set to) 12000.

Data collector. The data collector receives the reports from the probes on UDP port 12000 and every 20 seconds prints on screen *the top five TCP hitters*, defined as the 4-tuples (SOURCE IP, DESTINATION IP, SOURCE PORT, DESTINATION PORT) that carried the highest volume of traffic (TCP net data only) in the last observation window (of 20 seconds).

1. Write a C++ program that implements the traffic probe.
2. Write a C++ multi-threaded program that implements the data collector.

Note: all library functions from the NPL repository developed in the course can be used, as well as your favorite external JSON library (e.g., the `nlohmann` library).

NETWORK PROGRAMMING LABORATORY

18 July 2022

Exercise

In the following we will implement a networked version of the *fortune* application, in which a *fortune client* requests for a fortune sentence to a (possibly remote) *fortune server*. The application is supposed to run over UDP by using an arbitrary port (e.g. 12000).

Fortune client. Initiates the communication by sending a `fortune request` message to the server. The message is formatted as JSON and carries the `type` of message set to `REQ` and the (empty) `text` field. Then, the client waits for the message reply from the server. Once received, the message is parsed and printed to screen.

Fortune server. Listen for client requests. When a request is received, it prepares a *fortune* message and sends it back to the client. Again, the message is formatted as JSON with `type = REP` and the `text` field filled with the fortune sentence.

1. Write a C++ program that implements the fortune server. For your convenience, the `2022-07-18.hpp` header file is available in the exam folder with a few utilities functions (for JSON translation and fortune text crafting).
2. Write a C++ program that implements the fortune client.

Note: all library functions from the NPL repository developed in the course can be used, as well as your favorite external JSON library (e.g., the `nlohmann` library).

NETWORK PROGRAMMING LABORATORY

18 November 2022

Exercise

In the following you will implement a simple monitoring application for Linux to investigate how many IP packets and TCP segments carry the option field filled. You are requested to develop the traffic probe in two different ways.

Single Traffic Processor. A working thread captures the traffic and updates a stats record reporting:

- the number of IP packets carrying the option field filled
- the number of TCP segments carrying the option field filled
- the number of IP packets seen
- the number of TCP segments seen

A second thread reads the statistics and prints on screen every 5 seconds the percentages of IP and TCP packets with non empty option field.

Multi-Worker Traffic Processor. The traffic is distributed to a set of N working threads. Each thread captures traffic and updates its own stats record¹ reporting:

- the number of IP packets carrying the option field filled
- the number of TCP segments carrying the option field filled
- the number of IP packets seen
- the number of TCP segments seen

An additional thread reads the statistics and prints on screen every 5 seconds the percentages of IP and TCP packets with non empty option field for each thread as well as the aggregate values.

1. Write a C++ program that implements the *Single Traffic Processor* on the traffic captured from a network interface. Use one of the available network I/O frameworks (e.g., `AF_PACKET`, `pcap`) at will.
2. Write a C++ program that implements the *Multi-Worker Traffic Processor* on the traffic captured from a network interface. In this case you must use the `AF_PACKET` socket. Why?

Note: all library functions from the NPL repository developed in the course can be used.

¹Note: each working thread has its own stats record

NETWORK PROGRAMMING LABORATORY

30 January 2023

Exercise

In this project it is requested to implement the `find-my-ip` service, a simple client-server application to retrieve the IP address in use. This can be useful when the host is behind a NAT and needs to find out the public IP address provided by the ISP.

Client. The client program contacts the remote server by using the appropriate transport protocol over a predefined port. It is up to the implementer to choose the most suitable transport protocol and to define (if any) the format of the message request. Upon receiving the server response, the server parse the message and prints to standard output its own IP address.

Server. The server listens for client requests over the predefined port (and the selected transport protocol) and sends back to the client a JSON message containing the client IP address. Optionally, the server may add other information (client port number, time, etc.).

1. Write the C++ program `find-my-ip.cpp` that implements the client.
2. Write the C++ program `find-my-ip-server.cpp` that implements the server.

Note: all library functions from the NPL repository developed in the course can be used, as well as your favorite external JSON library (e.g., the `nlohmann` library).

NETWORK PROGRAMMING LABORATORY

5 June 2023

Exercise

A *Bloom filter* is a probabilistic data structure used to test whether an element is a member of a set $\mathcal{S}$. It uses a bit array of size N and a set of K hash functions to store and retrieve data efficiently. When an element is inserted into the filter, the hash functions generate a set of indexes in the bit array, and those indexes are marked as "true." To check if an element is in the set, the hash functions are again applied, and if any of the corresponding indexes in the bit array are "false," the element is determined to be not in the set. However, there is a small probability of false positives, where the filter may mistakenly report an element as being in the set when it is not. Bloom filters are commonly used in applications where approximate answers or fast membership queries are acceptable, such as caching, spell checking, and network routers.

Hash functions. A simple way of producing K hash functions is to use two hash functions $f(x)$ and $g(x)$ and combine them linearly, as:

$$h_k(x) = f(x) + k \cdot g(x) \pmod{N}, \quad k = 0, 1, \dots, K-1$$

From the theory, if n elements belong to the set $\mathcal{S}$ and N is the size of the Bloom filter, then the *optimal* value of K is:

$$K = \frac{N}{n} \ln(2).$$

Bloom filter C++ class. Write a C++ class (or structure) to implement a Bloom filter of size N , equipped with K hash functions. The class must implement the `insert(x)` method to add the element x to the set, and the `query(x)` method which returns `true` if the element x belongs to the set (`false`, if not).

Hint: Define the class template over the values N and K . Which internal member would you use?

Use the plain integer representation of IP addresses. Although not very efficient and well performing, the functions $f(x)$ and $g(x)$ can be simply selected as the *modulo* N integer representation of the IP address and any of its rearrangement (e.g., by swapping the sixteen upper and lower digits).

An approximate IP counter application. We want to use a Bloom filter to obtain an approximate count of the number of *different* IP destinations found in the provided pcap trace. To do so, write a C++ application in which you are requested to:

1. dimension a Bloom filter for a set of around 5K elements;
2. go through the packet trace and insert new IP addresses in the BF;
3. increment the counter only if the IP destination is new;
4. at the end, print the estimated count and compare it with the exact one (how do you get this value?)

NETWORK PROGRAMMING LABORATORY

3 June 2024

Exercise 1

TCP flows can be uniquely identified by the canonical four-tuple `<srcip, srcprt, dstip, dstprt>`. However, TCP traffic is inherently bi-directional; hence packets travelling in the reverse direction should be tagged with the same identifier. In C++, the type `tcpflowid_t` type can be defined as:

```
struct tcpflowid_t {
    uint32_t srcip;
    uint16_t srcprt;
    uint32_t dstip;
    uint16_t dstprt;
};
```

The C++ `unordered_map` container implements a collision hash table. In order to use a key of type `tcpflowid_t` it is then necessary to specialize the template classes `std::hash` and `std::equal_to`. This can be done by implementing both the *call* and *bool* operators in the following code:

```
namespace std {
    template <>
    struct hash<tcpflowid_t> {
        size_t operator()(const tcpflowid_t& f) const {
            //
            // Implement the hash function here
            //
        };
    };

    template<>
    struct equal_to<tcpflowid_t> {
        bool operator()(const tcpflowid_t& lhs, const tcpflowid_t& rhs) const {
            //
            // Implement the bool operator here
            //
        }
    };
}
```

The file `20240603.hpp` contains the definition of `tcpflowid_t` as well as the skeleton for the above classes specialization.

1. Fill in the call `std::hash<tcpflowid_t>::operator()` to produce a *symmetric* hash function, i.e. a function that returns the same value if applied to flow identifiers like `tcpflowid_t(x,y,z,w)` and `tcpflowid_t(z,w,x,y)`. *Hint: using the XOR logical operator may help a lot here...*
2. Fill in the bool `std::hash<tcpflowid_t>::operator()` to compare two `tcpflowid_t` objects. Again, remember that `tcpflowid_t(x,y,z,w)` and `tcpflowid_t(z,w,x,y)` identifies the same TCP flow.
3. Compile and run the test program `one.cpp` to check the correctness of the above implementations.

Exercise 2

TCP is a connection oriented protocol, hence any connection must be established before sending/receiving data. The following connection establishment process is known as *three-way handshake*:

- the client sends a segment with the **SYN** flag set to 1;
- the server responds with a segment in which both the **SYN** flag and the **ACK** flag are set to 1;
- finally the client concludes the handshake by sending a segment with the **ACK** flag set to 1.

Conversely, TCP connections can be either *abruptly* or *gracefully* closed. In the *abrupt* scenario, it is sufficient for one of the two peers to send a segment with the **RST** flag set to 1 to teardown the connection. In the *graceful* scenario, instead:

- one of the two peers (say P_1) sends a segment with the **FIN** flag set to 1;
- the other peer (say P_2) responds with a segment with the **ACK** flag set to 1 (*half close*), and then continues sending segments (if any);
- when P_2 has no more data to transmit, it also sends a segment with the **FIN** flag set to 1;
- P_1 then acknowledges receiving the **FIN** segment by responding with a segment with the **ACK** flag set to 1.

In this exercise, it is requested to implement a simple *TCP connection tracker* to detect full TCP connections, from their initial setup to their closing. To process of exactly tracking a TCP connection can be very tedious and it involves checking sequence/ack numbers, detecting retransmissions and so on. However, since our goal is to track full connections only, the whole process can be greatly simplified as it follows.

- The TCP connection is set as open once the very first segment carrying a **SYN** flag (but not the **ACK** flag) is found.
- The TCP connection is set as closed once the first segment carrying an **ACK** flag after two segments carrying a **FIN** flag is found.

The above mechanism is clearly approximated as it does not check sequence numbers and does not account for retransmissions, out-of-order segments, etc.. However, for our purposes, the algorithm provides reasonable estimates of the traffic volume exchanged by the two peers, of the completion time of connections and their throughput.

1. Based on the above described mechanism, write an application that reads a pcap file trace and extracts the *completed* TCP connections. For each of them, report:
 - the number of packets exchanged between the two TCP peers
 - the number of bytes (TCP payload data only) exchanged between the two TCP peers
 - the completion time of the connection
 - the connection throughput
2. (Extra) Extend the tracker app to perform TCP connection tracking on live capture captured from a physical network device.

NETWORK PROGRAMMING LABORATORY

13 June 2025

Exercise

The **Domain Name System (DNS)** is a distributed hierarchical system that resolves human-readable domain names (e.g., `www.example.com`) into IP addresses (e.g., `93.184.216.34`). A DNS query typically involves:

- A client sending a query to a recursive DNS server.
- UDP port 53 is used for most DNS queries (TCP is used for larger packets or DNSSEC).
- The server replies with an answer or a referral.

A DNS packet contains a header, questions, and (optionally) answers, authorities, and additional information.

In any case, each query contains:

- A domain name
- A query type (QTYPE): e.g., A, CNAME, NS, MX, AAAA, etc.

DNS Query Packet Format (for reference)

		DNS Header (12 bytes)					
		Query Name (variable length)					
		Query Type (2 bytes)		Query Class (2 bytes)			
		Answer Section (optional)					
		Authority Section (optional)					
		Additional Section (optional)					

Write a Rust program that captures or analyzes DNS queries from network traffic and logs detailed information and statistics.

Command-Line Interface

Use the `clap` crate to implement the CLI with:

- `--interface` or `-i`: Capture live packets on the selected interface (e.g., `eth0`)
- `--pcap-file` or `-f`: Analyze packets from an offline PCAP file
- `--interval` or `-n`: Time interval in seconds to print stats (default: 5)

Only one of `--interface` or `--pcap-file` may be used at a time. The program should print:

- For live mode: [*] Reading live packets from interface 'eth0'...
- For offline mode: [*] Reading packets from file 'trace.pcap'...

DNS Query Packet Processing

Use the `pcap` crate to handle packets:

- In live mode: Open the interface in promiscuous mode and apply the proper BPF filter
- In offline mode: Open the PCAP file and apply the same filter
- For each packet:
 - Parse the UDP payload as a DNS packet using `simple-dns`
 - * The `Packet::parse()` method requires the DNS payload only as input
 - For each question:
 - * Collect *query name* and *query type* for logging and statistics

Output and Statistics

In live mode, the program must print the statistics every `n` seconds (from `--interval`). In the offline mode, stats are printed once at the very end, when processing is completed. The output should include:

- Current timestamp
- **Query Type Counts** (only A, CNAME, MX, NS, Other)
- **Top 5 Domain Names** queried, with counts, in descending order

Example:

```
=== DNS Stats at 14:52:03 ===
```

```
Query Type Statistics:
```

```
A: 15
```

```
CNAME: 8
```

```
Top 5 Queried Domains:
```

```
1. www.google.com - 6
```

```
2. api.github.com - 4
```

```
3. www.example.com - 2
```

```
4. ftp.example.org - 1
```

```
5. docs.rust-lang.org - 1
```